The University of Sydney

School of Computer Science

COMP5318 Assignment 2

A Group Assignment submitted for the UoS:

COMP5318 Machine Learning and Data Mining

Group No. A2 group 87

Group member: cliu0947 (540584226)

llai0711 (540202537)

Date: May 5, 2025

Table of Contents

1. Introduction *1*
2. Data 1
2.1. Data description and exploration 1
2.1.1 Observe the information of dataset 1
2.1.2 Observe characteristics of the images 1
2.2. Pre-processing 3
2.2.1 Normalization and Gaussian Bluring 3
2.2.2 PCA 4
3. Methods *6*
3.1. Random Forest Classifier 6
3.1.1 Theory 6
3.1.2 Strengths and Weaknesses 6
3.1.3 Architecture and hyperparameters 6
3.2 Multilayer Perceptron Neural Network 8
3.2.1 Theory 8
3.2.2 Strengths and Weaknesses 9
3.2.3 Architecture and hyperparameters 9
3.3 Convolutional Neural Networks 12
3.3.1 Theory 12
3.2.2 Strengths and Weaknesses 13
3.2.3 Architecture and hyperparameters 13
4. Results and discussion *16*
4.1 Random Forest Classifier 16
4.2 Multilayer Perceptron Neural Network 16
4.3 Convolutional Neural Networks 18
4.4 Comparison of the models 20
5. Conclusion *22*
5.1 Summary of main findings and identification of study limitations 22
5.2 Future Work Suggestions 22
6. Reflection *23*
6.1 Student SID: 540584226 23
6.1 Student SID: 540202537 23

1. Introduction

Colon Pathology is a field of medicine that studies the behaviour of colon tissue in health and disease [1]. Pathologists usually confirm tumour types by analysing pathological slide images. However, the manual reading process is time-consuming and easily affected by the pathologists' subjective judgment. With the rapid development of machine learning and deep learning technologies, automatic analysis of pathological images has become feasible [2]. In this study, we use the PathMNIST dataset to train the model to build a classification model for automatic analysis of pathological images. We aim to train a model to assist doctors in disease diagnosis, reduce their workload, and improve diagnostic efficiency and accuracy.

The PathMNIST dataset contains 40000 well-labelled and high-quality images, which is helpful for training the model. This dataset is derived from real images of human tissue slices [3]. This dataset has been widely used for model training. We can compare and analyse our model with those developed by others, thereby enhancing the model's performance.

This study is based on the PathMNIST dataset and systematically compares three types of algorithms: Random Forest, CNN and MLP. By comparing these three algorithms in the same environment and dataset, we ensure a fair comparison and enhance the effectiveness of conclusions. In addition, we use random search to tune hyperparameters for the above three algorithms. Comparing all algorithms after hyperparameter tuning can improve fairness and reduce bias. Different algorithms have varying degrees of sensitivity to hyperparameters. If we don't conduct hyperparameter tuning, some algorithms may be underestimated due to poor parameter settings. Finally, we select the optimal model for each algorithm based on the results. We found that CNN has higher accuracy than MLP and Random Forest. Finding a highly accurate model can help doctors reduce misdiagnosis and missed diagnosis.

2. Data

2.1. Data description and exploration

2.1.1 Observe the information of dataset

The dataset utilized in this project is PathMNIST. The PathMNIST dataset consisted of hematoxylin & eosin-stained histological images. In the given dataset, train and test data include 32000 and 8000 RGB images, respectively. In addition, each image is resized to 28 × 28 pixels. The dataset was divided into three subsets: 25,600 images in the training set, 6,400 images in the validation set, and 8,000 images in the test set. Each dataset is comprised of 9 target tissue categories. The main task is to design multiple algorithms to classify medical images.

This dataset contains 9 categories, and there are differences in the number of samples between categories. The sample sizes for each category are 3490, 3431, 3505, 3656, 2950, 4290, 2728, 3253, and 4697, respectively.

2.1.2 Observe characteristics of the images

In order to better understand the PathMNIST dataset and provide support for subsequent preprocessing and modeling processes, we analyzed the image features from the following aspects:

Firstly, Figure 1 shows that the sample images have a low resolution. However, important histological features are preserved. Moreover, H&E staining lets the dominant hues become purple and pink, which may cause a colour bias.

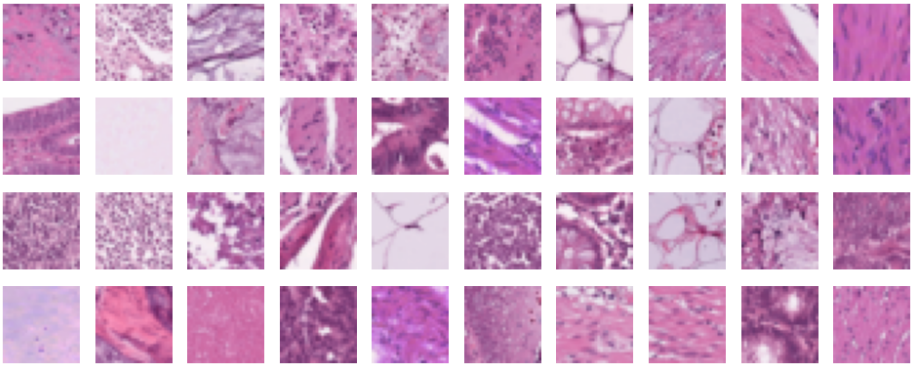

Figure 1. Example of original training data

Secondly, Figure 2 reveals that interest distributions are vertically clustered between widths 12 and 15 and horizontally distributed between heights 12 and 16. This central bias indicates that the features are generally aligned but not perfectly centred. It might slightly impact the model's sensitivity to spatial variations. Furthermore, the outliers in both the X and Y axes may be noise during model learning. We need to apply denoising in preprocessing.

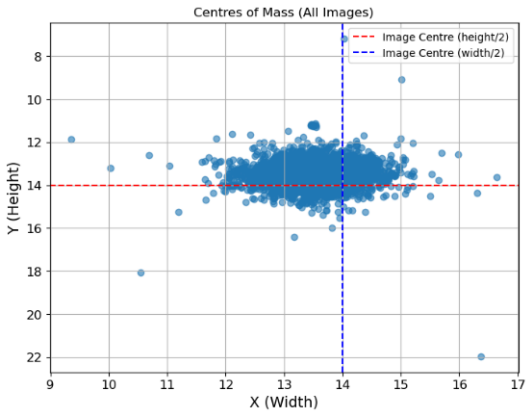

Figure 2. Centre of Mass Distribution

Thirdly, Figure 3 demonstrates a right-skewed distribution, where most features occupy approximately 250 to 500 pixels. This means that major informative regions in the images are moderately sized. Nonetheless, the tail in larger sizes indicates some images with significantly larger features. We need to implement normalization in preprocessing.

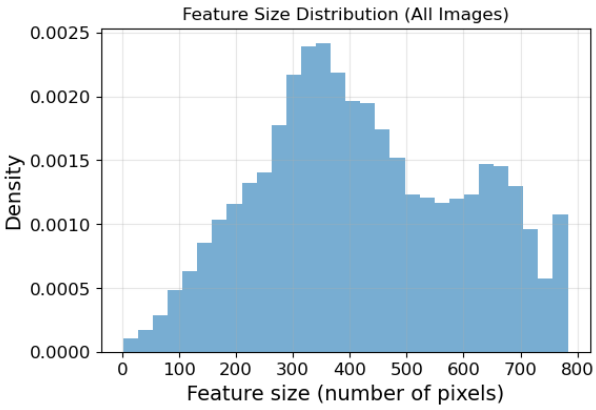

Figure 3. Feature Size Distribution

Fourthly, Figure 4 shows values that range from around 80 to 250, with a peak of about 160. These peaks align with the typical staining characteristics of hematoxylin and eosin (H&E), which produces characteristic purple and pink hues. The extremely high peak suggests that a specific colour is dominant across the dataset. This distribution poses a risk of overfitting. If the model learns to rely on colour patterns rather than image features.

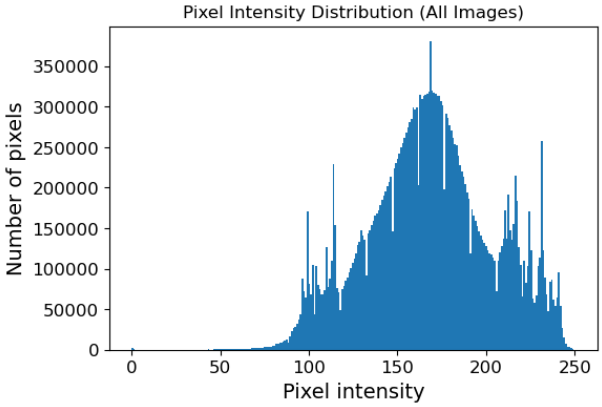

Figure 4. Pixel Intensity Distribution

Fifthly, Figure 5 reveals that classes 3 and 6 have similar appearances. It might make it difficult for algorithms to classify.

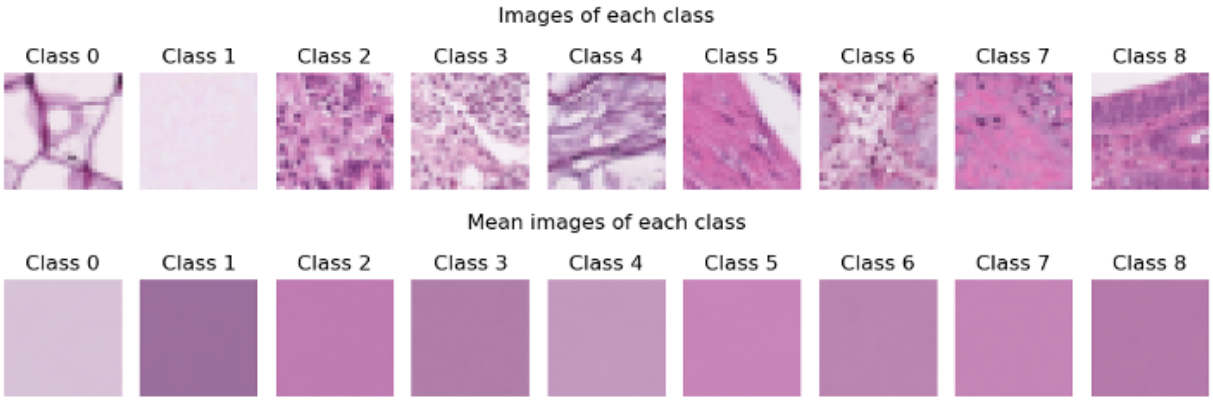

Figure 5. Sample and Mean images of each class

Overall, this exploration reveals that the PathMNIST dataset is a challenging dataset in medical image classification, it includes colour bias, non-centred features, and feature size variation. To address these issues, we implement several preprocessing methods in the next section.

2.2. Pre-processing

2.2.1 Normalization and Gaussian Bluring

Based on several insights from data exploration, two preprocessing techniques were applied to enhance model performance.

- Gaussian Blurring

  Due to the outliers present in the centre of mass distribution, we applied Gaussian Blurring with a kernel size (3,3) and a sigma value of 0.5 to the train and test datasets. In this way, images with high-frequency noise were reduced, while the overall structure of tissue features was preserved. In addition, the model can focus more on relevant spatial patterns.

- Z-score Normalization

  To adjust large feature sizes in the feature size distribution, we applied Z-score normalization on all images. First, we calculated the mean and standard deviation of all images in the train and test datasets. Then, we normalized each image by subtracting the mean and dividing by the standard deviation. Moreover, we added a small constant 1e-5 to prevent division by zero. The following formula shows Z-score generation:

$$Z = \frac{x - \mu}{\sigma + \varepsilon}$$

  Where Z is the Z-score of images, x is the original pixel of image, μ represents the mean pixel values of image, σ represents the standard deviation of image, and ε represents the small constant. In this way, normalization can improve the model's convergence behaviour and generalization performance.

For normalization, we used to choose Min-Max Normalization. This method rescales pixel values to a fixed range [0, 1]. It is a commonly used normalization technique for image preprocessing, and it was introduced in the tutorial. We applied this method initially. However, the MLP model showed very low accuracy. Therefore, we changed to Z-score Normalization. The MLP model improved the performance significantly with higher validation accuracy. Z-score normalization standardizes each image to zero mean and unit variance. Since it produces zero-centered inputs, it is not only effective for deep learning models but also improve compatibility with activation functions ReLU.

Overall, these preprocessing methods are chosen based on data exploration analysis. In medical images, we need to preserve complete structure but minimise irrelevant noises. Preprocessing techniques such as normalization and noise reduction are important on medical image classification tasks [4]. Gaussian blurring helps smooth tissue boundaries without erasing them. Z-score Normalization standardizes image intensity distributions. They all can improve medical image classification tasks.

2.2.2 PCA

PCA transforms original data into a new space with principal components. Principal components are vectors that define a new coordinate system. These principal components are ordered according to the amount of variance they capture. The components with higher variances capture a higher variability of the data. PCA examines the covariance relationship of the data to find principal components and selects the top k principal components that capture the most variance within the dataset. Then PCA projects the original data onto the lower-dimensional subspace formed by the selected principal components. The resulting dataset can be used as an input to train a ML algorithm. PCA can reduce redundant and highly correlated features in the data, but it may also neglect some detailed information [5].

We observed the cumulative explained variance by number of principal components (Figure 6). We found that about 100 principal components can explain over 90% of the variance, and about 200 principal components can achieve 95% of the variance. When the number of principal components is greater than 500, the curve is almost horizontal. To reduce the dimensionality of the data while retaining most information, we reduce the feature dimensionality from 2352 to 526 while retaining at least 99% of the total variance.

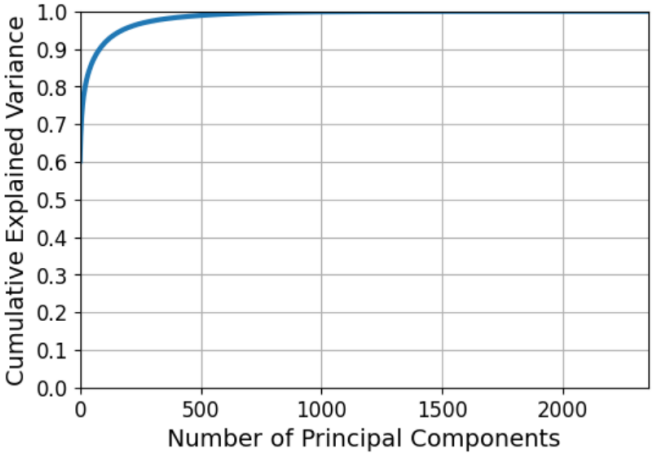

Figure 6. Cumulative Explained Variance by Number of Principal Components

But PCA has some weaknesses. PCA sort the principal components based on the amount of variance they captured. But principal components with high variance do not always capture useful information for subsequent tasks. Therefore, features that contain key information but have small variances may be discarded. Such information loss may have a negative impact on the performance of subsequent modelling. The original features are transformed into a new set of principal components through PCA, but these principal components usually lack clear physical or semantic interpretations. Therefore, the interpretability of the components is reduced. In addition, due to PCA discarding some principal components during the dimensionality reduction process, the information in the original data is partially lost. Therefore, the reduced dimensional data cannot fully recover the original data, only an approximate of the original data can be reconstructed. Figure 7 shows the original and pre-processed images in train and test data.

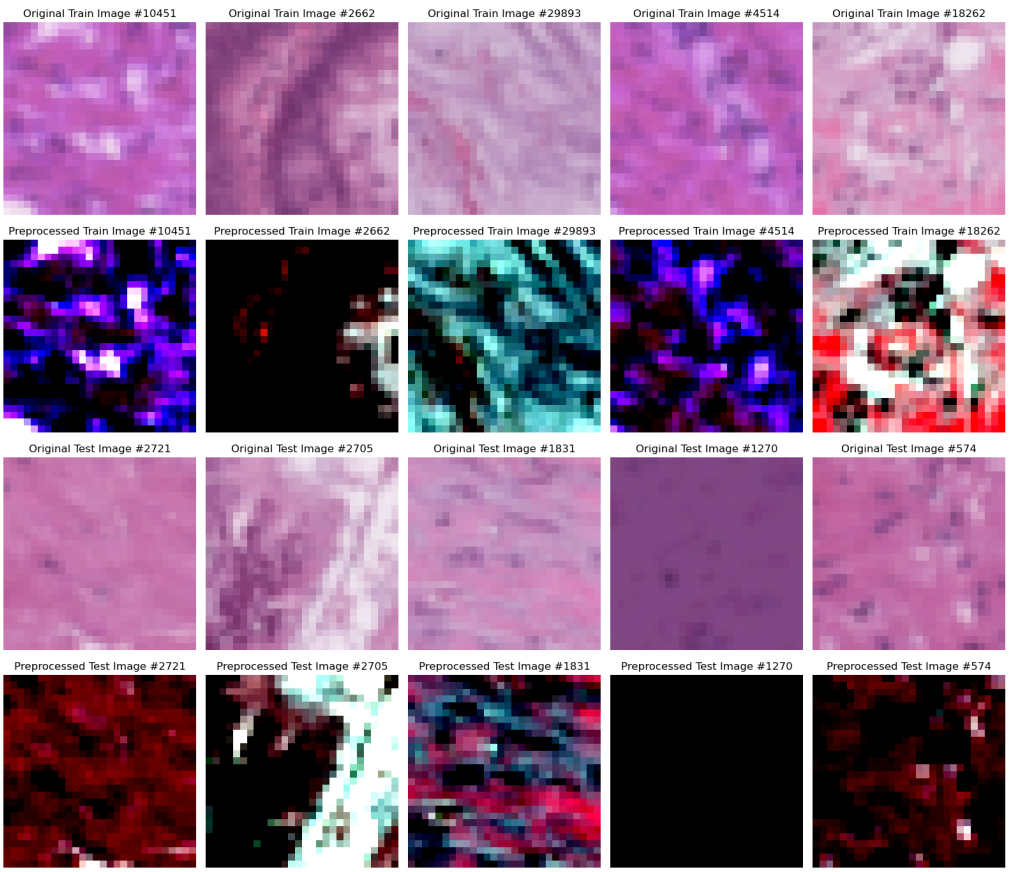

Figure 7. Examples of original and preprocessed data

3. Methods

3.1. Random Forest Classifier

3.1.1 Theory

Random Forest is an ensemble method based on Bagging. Random Forest performs bootstrap on the training set to provide different training subsets for each tree. When each tree node splits, a portion of candidate features is randomly selected from all features for optimal partitioning. Because each base classifier in the Random Forest is trained by a subset of the features, Random Forest can effectively reduce the correlation between decision trees, which helps to decrease model variance and prevent overfitting. Finally, the results are obtained through majority voting for classification tasks. Random Forest mitigates the instability and sensitivity of individual decision trees, and is robust to overfitting [6].

3.1.2 Strengths and Weaknesses

Random Forest compromise of multiple independent decision trees. It makes predictions by integrating multiple independent decision trees.

- Strengths:

  Each decision tree in the Random Forest is trained on different samples and subsets of features, resulting in a low correlation and diversity between each decision tree. Decorrelation can reduce the prediction errors of each decision tree and decrease the variance of the model. Hence, Random Forest is robust to overfitting. The voting mechanism of Random Forest reduces the influence of individual trees affected by noise, leading to enhance robustness of the overall model to noise and outliers.
- Weaknesses:

  Random forest is an ensemble model composed of different decision trees. Each decision tree structure may be different. As a result, the structure of Random Forest is complex and difficult to understand intuitively. In addition, since each tree only uses partial features, and the overall result is determined by the combination of multiple decision trees. Therefore, Random Forest lacks interpretability. Random Forest requires traversing all decision trees for each prediction, which is much slower than a single decision tree. In regression tasks, the final output of Random Forest is the average of all decision tree predictions. Random Forest often underestimates extreme high values and overestimates extreme low values. Random Forest treats each pixel as an independent feature and cannot understand the spatial relationships between these pixels. Therefore, it lacks the ability to learn high-level semantic representations from the original image.

3.1.3 Architecture and hyperparameters

We trained a Random Forest model. We conducted two experiments to select a suitable hyperparameters configuration of Random Forest (Table 1 and Table 2). After the two experiments, we implemented the Random Forest hyperparameter configuration as follows:

- Criterion='entropy': Using information gain to select the optimal splitting point. Compared to using Gini as the splitting criterion, using Entropy as the splitting criterion has higher accuracy but also longer training time (Table 1). We select Entropy to gain a higher accuracy.
- max_features='sqrt': At each leaf node split, the node randomly selects a subset of $\sqrt{n}$ features to reduce overfitting. Compared to using log2 as the max_features, using sqrt as the

max_features has higher accuracy but also longer training time (Table 2). We select sqrt to gain a higher accuracy.

- n_estimator=300: Random Forest is comprised of 300 decision trees. 300 decision trees can ensure the accuracy of the model while training it relatively quickly.
- max_depth=None: Letting each decision tree grow fully to increase fitting accuracy.
- min_samples_split=5: Setting a node of the decision tree must have at least 5 samples before splitting, which reduces overfitting risk and enhances training speed.
- min_samples_leaf=4: Setting a leaf node of the decision tree must have at least 4 samples to reduce overfitting, which reduces sensitivity to noise and risk of overfitting.

Table 1. Experiment One: Comparison of Random Forest performance under different splitting criteria

Criterion	Training Time	Accuracy on test set
Entropy	1.5 sec	0.562
Gini	0.7 sec	0.536

Table 2. Experiment Two: Comparison of Random Forest performance under different values of max_features

Max_features	Training Time	Accuracy on test set
Sqrt	0.49 sec	0.507
Log2	0.28 sec	0.471

We tune the four hyperparameters: n_estimators, max_depth, min_Samples_split, and min_Samples_leaf (Table 3). The value of n_estimators controls the number of decision trees in a random forest. The value of max_depth controls the complexity of each decision tree. The value of min_samples_split and min_samples_leaf controls how decision nodes are created. By tuning these four hyperparameters, we can optimize the Random Forest model. We employed random search combined with stratified 10-fold cross to optimize the four selected hyperparameters. We first set the range of values for these four parameters. Then we use RandomSearch to randomly search for the 20 combinations of hyperparameters. Subsequently, we performed 10-fold stratified cross-validation on each combination to evaluate its mean accuracy on the validation set. We listed the five highest-scoring combinations after the search is complete.

Table 3. Random Forest Hyperparameters for tuning

Hyperparameters	Values	Description	Expected Effect
n_estimators	200, 300, 500	The number of decision trees in a random forest	More decision trees generally improve generalization and increase training time. But when there are too many decision trees, the marginal returns decrease

max_depth	None, 15, 30	The maximum number of layers allowed for a single decision tree	Greater decision tree depth generally increases the model's capacity and increases risk of overfitting. However, excessive tree depth can make it easier for a single decision tree to fit noise
min_samples_split	2, 5, 10	The minimum sample size required for the nodes of a decision tree to continue splitting	A larger value of min_samples_split generally reduces the risk of overfitting and accelerates training but may increase bias. A small value of min_samples_split increases the model's capacity, but increases the risk of overfitting
min_samples_leaf	2, 4, 8	The minimum number of samples retained by leaf nodes in a decision tree	A larger value of min_samples_leaf generally enhances robustness against noise, but may cause the model to overlook subtle patterns. A small value of min_samples_leaf increases the model's capacity, but increases the risk of overfitting

3.2 Multilayer Perceptron Neural Network

3.2.1 Theory

Multilayer Perceptron Neural Network (MLP) is a type of feedforward neural network. The network is composed of an input layer, several hidden layers, and an output layer connected in series. Input Layer is responsible for receiving original data. Hidden Layers is responsible for feature transformation and extraction. The output layer is responsible for providing the final prediction results. Each neuron in the current layer is connected with all neurons from the previous and no connections within the same layer. A single neuron calculates the weighed sum of the outputs of the previous layer, which is passed through a transfer function. The transfer function needs to be differentiable and it usually using nonlinear activation functions. The calculation process of the entire network is forward propagation, where data passes through various hidden layers from the input layer and finally reaches the output layer. MLP usually uses backpropagation algorithm for training. The backpropagation algorithm calculates the output error and then uses the chain rule to backpropagate the gradient layer by layer. The optimizer then updates the weights and biases based on these gradients. MLP is suitable for classification and regression tasks of structured data.

3.2.2 Strengths and Weaknesses

MLP is a feedforward neural network composed of multiple fully connected layers of neurons, capable of learning complex nonlinear relationships between input and output.

- Strengths:

  MLP is a feedforward neural network that applies linear transformation and non-linear activation functions. Each hidden layer's neuron can learn different feature combinations, and the superposition of these features can gradually approximate more complex function structures. MLP trained with the backpropagation algorithm are universal approximators. Any continuous function can be approximated with arbitrary small error by a network with one hidden layer [7]. Any function can be approximated to arbitrary small error by a network with two hidden layers [8]. The forward and backward propagation processes of MLP consist of matrix multiplication and vector operations, which are easy to parallelization and hardware accelerate
- Weaknesses:

  The number of hidden layers and neurons in MLP needs to be reasonably controlled. When there are too few neurons, the model's expressive ability is insufficient, which can easily lead to underfitting. When there are too many neurons, the model has a large number of parameters, which makes it easy to remember the training data. As a result, it may lead to overfitting and slower training speed. MLP converts input into output through multiple hidden layers, each layer performance transformations on data. Moreover, MLP has many parameters, resulting in poor interpretability. Deep or wide neural networks have a large number of parameters and involve complex computations, leading to longer training time. MLP flattens the image into one-dimensional vectors and inputs them into the network, which results in MLP losing the spatial local relationships between pixels in the image. Therefore, MLP cannot utilize some local patterns in images for feature extraction.

3.2.3 Architecture and hyperparameters

Before implementing our MLP, we conducted several experiments first.

In experiment one, we compared the MLP performance under different layers. When the hidden layers increase from 3 to 4, the accuracy of the test set decreases from 0.644 to 0.61, and the training time increases from 4.25 seconds to 4.94 seconds. Therefore, we set the hidden layer to 3 layers. The experiment results are showed in Table 4.

In experiment two, we explored the impact of batch normalization of MLP performance. After enabling Batch Normalization, the accuracy on test set increased from 0.641 to 0.668 and the training time increased from 4.27 seconds to 6.56 seconds. The experiment results are showed in Table 5.

Table 4. Experiment One: Comparison of MLP performance under different hidden layers

Hidden Layer	Training Time	Accuracy on test set
3	4.25 sec	0.644
4	4.94 sec	0.610

Table 5. Experiment Two: Impact of Batch Normalization on MLP performance

Batch_normal	Training Time	Accuracy on test set
True	6.56 sec	0.668
False	4.27 sec	0.641

In addition to the two experiments presented in Jupyter Notebook, we also conducted several additional experiments to further optimize the MLP's hyperparameters. These experiments include exploring the effects of different activation functions, testing different learning rate settings and various combinations of hidden layer structures, and so on. Through these experiments, we set up the following MLP model (Figure 8 and Table 6).

The MLP model flattens 28 × 28 × 3 pixels into a 2352-dimensional vector input. It employs a three-layer fully connected structure with a decreasing number of neurons: 160 → 128 → 96. The first layer with a large number of neurons can capture the complex relationship of features. By reducing the number of neurons layer by layer, redundant information can be reduced, and training speed can be improved. To mitigate overfitting, the dropout rate of the first hidden layer is set to 0.2, and the dropout rate of the second and third hidden layers is set to 0.1. In order to obtain higher accuracy, we chose to enable Batch Normalization. In addition, to improve MLP stability and performance, we employed Batch Normalization on each hidden layer. Moreover, we address the problem of gradient vanishing by setting the activation function to Relu. Due to the dataset containing 9 categories, the output layer is set to 9 units with a Softmax activation. The MLP model is trained using the Adam optimizer with a learning rate set to 0.0001. Using Adam can automatically adjust the learning rate, with fast convergence and stable performance.

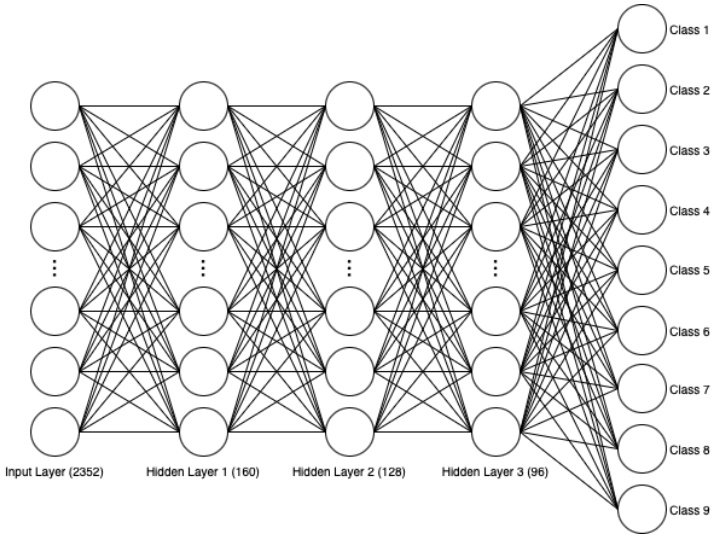

Figure 8. MLP architecture

Table 6: Detailed architecture and parameter count of MLP

Layer (type)	Output Shape	Param #
flatten_18 (Flatten)	(None, 2352)	0
dense_1 (Dense)	(None, 160)	376480
bn1 (BatchNormalization)	(None, 160)	640
dropout_1 (Dropout)	(None, 160)	0
dense_2 (Dense)	(None, 128)	20608
bn2 (BatchNormalization)	(None, 128)	512
dropout_2 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 96)	12384
batch_normalization_3 (BatchNormalization)	(None, 96)	384
dropout_3 (Dropout)	(None, 96)	0
output (Dense)	(None, 9)	873

Total params: 411,881
Trainable params: 411,113
Non-trainable params: 768

We tuned the four hyperparameters: base units, dropout rate, L2 regularization, and learning rate (Table 7). Base unit controls the overall size of the network. Dropout rate controls the proportion of randomly deactivating neurons in each forward propagation during the training process. L2 regularization controls the magnitude of model weights and suppresses excessive weights. Learning rate controls Adam's update amplitude at each step. By tuning these four hyperparameters, we can optimize the MLP model. We first set the range of values for these four parameters. Then we used RandomSearch to randomly search for 20 combinations of hyperparameters. These combinations of hyperparameters were trained on a fixed training set, with validation set accuracy as the target metric for single training evaluation. Finally, we listed the top 5 combinations with the highest scores.

Table 7. MLP Hyperparameters for tuning

Hyperparameters	Values	Description	Expected Effect
Base unit	128, 192, 256, 320, 384, 448, 512	Number of neurons in the first hidden layer. The last two hidden layers decrease by 1/2 and 1/4 respectively	A larger number of units can capture complex features, but it may lead to overfitting. In addition, A larger number of units will increase computational costs. If the number of units is small, the model capacity will be reduced, which may result in underfitting
Dropout rate 0	0.1, 0.2, 0.3, 0.4, 0.5	Dropout rate in the first hidden layer. It prevents overfitting by randomly deactivating neurons.	Using dropout can improve model robustness and generalization. As the Dropout rate increases, the model's dependence on specific neuronal pathways weakens. A Dropout rate that is too high may miss information, which leads to underfitting. A dropout rate that is too low will weaken its regularization effect, which increases the risk of overfitting
Dropout rate 1	0.1, 0.2, 0.3, 0.4, 0.5	Dropout rate in the second hidden layer. It prevents overfitting by randomly deactivating neurons	Using dropout can improve model robustness and generalization. As the Dropout rate increases, the model's dependence on specific neuronal pathways weakens. A Dropout rate that is too high may miss information, which leads to underfitting. A dropout rate that is too low will weaken its regularization effect, which increases the risk of overfitting
Dropout rate 2	0.1, 0.2, 0.3, 0.4, 0.5	Dropout rate in the third hidden layer. It prevents overfitting by randomly deactivating neurons	Using dropout can improve model robustness and generalization. As the Dropout rate increases, the model's dependence on specific neuronal pathways weakens. A Dropout rate that is

			too high may miss information, which leads to underfitting. A dropout rate that is too low will weaken its regularization effect, which increases the risk of overfitting
L2 regularization	0.01, 0.001, 0.0001	L2 regularization adds the sum of squared weight parameters as a penalty term in the loss function to reduce the risk of overfitting	L2 regularization is used to control the complexity of a model. The larger the L2 regularization, the simpler the model and the lower the risk of overfitting, but it is prone to underfitting. The smaller the L2 regularization term, the stronger the model fitting ability, but it is prone to overfitting
Learning rate	0.01, 0.001, 0.0001	Learning rate controls the step size of each parameter update	As the learning rate increases, the step size of model parameter updates increases, and the convergence speed accelerates. A learning rate that is too high may cause oscillations or failure to converge, while a learning rate that is too low may result in slow training speed and may fall into local optima

3.3 Convolutional Neural Networks

3.3.1 Theory

Convolutional Neural Networks (CNNs) are designed to process data of multiple arrays, such as a colour image with three colour channels [9]. A CNN is composed of an input, output, and several hidden layers, including *Convolutional layers, Pooling layers, Flatten layers, and Fully Connected layers*. Convolutional layer applies numerous filters with local receptive fields to extract spatial features. Thus, they can detect patterns such as edges or textures in early layers and learn more complex patterns in deeper layers. Then, these features are passed through one-linear activations, such as ReLU. Pooling layer (e.g., MaxPooling) down-samples feature maps. It not only reduces the spatial dimensions of feature maps but also preserves important activations. This can not only improve computation efficiency but also provide spatial invariance. Flatten layer transforms the multi-dimensional input into a one-dimensional output, so that it can pass to Fully Connected layer. Fully Connected layer also called Dense layer that it combines the extracted spatial features and classify them into 9 classes by using softmax activation.

CNNs are designed for image recognition and classification tasks. Due to their hierarchical structure, they can capture spatial hierarchies from images and learn them effectively. Moreover, compared to fully connected networks, CNNs use weight sharing to the same filters across different spatial locations. These benefits can preserve local relationships and reduce the number of parameters by applying filters. In fully connected layers, dropout can be used to improve generalization and help prevent overfitting.

We initially implemented the VGG16 model. That's because VGG16 is a very deep convolutional network for large-scale image classification and is designed for 224×224 images [10]. However, our dataset contains 28×28 images. To let the VGG16 model fit image inputs, we modify the VGG16 model

to a smaller version. We can not only avoid excessive dimension reduction but also improve training efficiency.

3.2.2 Strengths and Weaknesses

Convolutional Neural Networks (CNNs) are designed to process spatial hierarchical data, such as images. Their architecture is based on three key principles: local connectivity, weight sharing, and hierarchical feature learning [9].

- Strengths:

  CNNs use local connectivity, which means each neuron is connected to a small receptive field. Hence, the network can focus on local spatial features, such as edges and textures. In addition, weight sharing applied the image with the same filter. Compared to a fully connected network and a random forest classifier, it can reduce the number of parameters. To explore spatial structure and share weights across spaces. Therefore, CNNs are not only more computationally efficient but also less chance to overfitting. In addition, they can learn hierarchical features. For example, they can learn from low-level patterns in early layers to more abstract patterns in deeper layers. Furthermore, CNNS are translation-invariant, so they can recognize features no matter their position in the image.
- Weaknesses:

  Due to the high capacity of CNNs, they may overfit if they are trained on small datasets and without using regularization techniques, such as dropout. The dropout function randomly deactivates neurons in training. By doing so, the network tends to learn more general features and prevent from co-adoption. Furthermore, deeper CNN architectures have higher computation costs and longer runtimes. For instance, the VGG16 model needs to handle large numbers of inputs and pass through numerous layers. The input images of this dataset may over-compress spatial information. Although pooling layers help CNNs reduce computation cost and improve spatial invariance. But excessive pooling layers may compress feature maps too much, losing critical features. For example, 28×28 images passing through 5 max pooling layers would reduce the dimensions: $28 \rightarrow 14 \rightarrow 7 \rightarrow 3 \rightarrow 1 \rightarrow 1$.

To address these issues, we reduced the number of convolutional blocks in the original VGG16 architecture. As a result, CNNs can preserve the feature maps and improve the running time. On the other hand, MLP has weaker performance for image classification. Because it loses spatial context by flattening inputs at first. Overall, CNNs have better theoretical and practical benefits for image classification tasks.

3.2.3 Architecture and hyperparameters

Before we built a CNN architecture, we conducted several experiments to justify our decisions in the theory section.

Firstly, we want to find out which encoding methods are more efficient. We conducted an experiment that encoded labels to on hot or integer encoding. We trained each encoding method for 10 epochs. Table 8 shows the results.

Table 8. Experiment: Comparison of CNN under different encoding methods

Encoding methods	Training Time	Accuracy on test set
One Hot Encoding	57.3 sec	0.8816
Integer Encoding	43.5 sec	0.8936

Secondly, we conducted another experiment that fitted a VGG16 model to this dataset. We trained this model for 5 epochs. After training 5 epochs, the training time was 127.7 sec, and the accuracy on test data was 0.777. We found that the VGG16 model cost a lot of time during training, and the accuracy was not even high enough. The reason is that the VGG16 model architecture was too large and deep, which caused excessive dimension reduction and information loss.

To address these issues, we implemented a simple version of the VGG16 model with only three convolutional blocks. This *"mini VGG16"* architecture retains essential feature extraction capabilities but reduces computation loads.

The *"mini VGG16"* architecture inputs fixed-size 28×28 RGB images. The image is passed through three convolutional blocks: the first two blocks contain 2 convolutional layers (32 and 64 filters, respectively), and the third block with 1 convolutional layer (128 filters). Each convolutional layer used filters with a small receptive field of 3×3. In addition, these convolutional blocks were followed by a 2×2 max pooling layer to reduce spatial dimensions. Max-pooling is set with a 2×2 pixel window and stride 2. These convolutional blocks are followed by 1 Fully Connected (Dense) layer with 128 units and 1 dropout layer with a rate of 0.5 to prevent overfitting. The final layer is a softmax output layer for classifying 9 classes. Moreover, all hidden layers are equipped with the activation function ReLU. Figure 9 and Table 9 shows the mini VGG16 architecture.

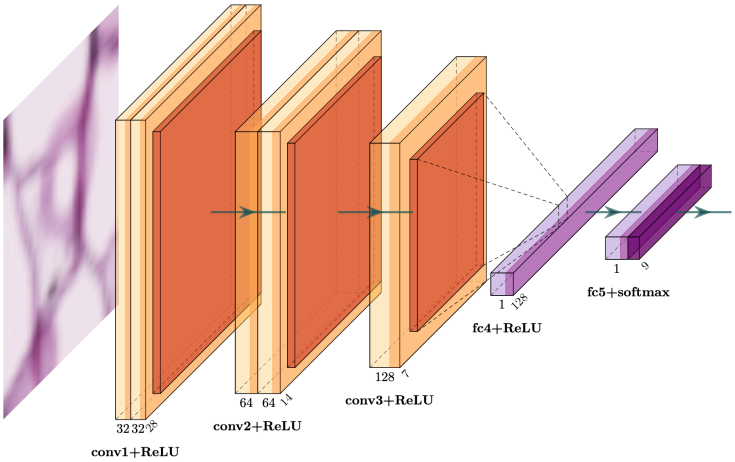

Figure 9. mini VGG16 architecture

Table 9. Detailed architecture and parameter count of the mini VGG16 model

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	[(None, 28, 28, 3)]	0
block1_conv1 (Conv2D)	(None, 28, 28, 32)	896
block1_conv2 (Conv2D)	(None, 28, 28, 32)	9248
block1_pool (MaxPooling2D)	(None, 14, 14, 32)	0
block2_conv1 (Conv2D)	(None, 14, 14, 64)	18496
block2_conv2 (Conv2D)	(None, 14, 14, 64)	36928
block2_pool (MaxPooling2D)	(None, 7, 7, 64)	0
block3_conv1 (Conv2D)	(None, 7, 7, 128)	73856
block3_pool (MaxPooling2D)	(None, 3, 3, 128)	0
flatten (Flatten)	(None, 1152)	0
dense_1 (Dense)	(None, 128)	147584
dropout_1 (Dropout)	(None, 128)	0
output (Dense)	(None, 9)	1161

Total params: 288,169
Trainable params: 288,169
Non-trainable params: 0

We selected four hyperparameters that they have a significant influence on CNN performance.

- Base filter size determines the number of filters in the first convolutional layer. In addition, the second and third convolutional layers are doubled and quadrupled, respectively. This parameter affects the capacity of the model to extract spatial features.
- Dense units control the number of neurons in the fully connected layer. This parameter influences the ability of the model to combine extracted features.
- Dropout rate defines the frequency of neurons being deactivated. This parameter is crucial for regularization to help the model prevent overfitting.
- Learning rate manages the size of the step moves at each epoch. This parameter directly affects the speed of the model's convergence during training.

We used *Keras tuner* to implement hyperparameter tuning. Table 10 shows the hyperparameters that we have selected. We use a random search running 20 trials, and each trial trains 10 epochs. In addition, we monitor the validation accuracy. We want to find out the best hyperparameter combinations that produce the highest validation accuracy.

Table 10. CNN Hyperparameters for tuning

Hyperparameters	Values	Description	Expected Effect
Base filter	32, 64, 96, 128	The number of filters in the first convolutional block. Second and third convolutional block's filters is double and quadrupled respectively.	Small filters may capture fewer spatial features. Large filters may cause overfitting or extract redundant information.
Dense units	64, 128, 192, 256	The number of units in fully connected layer. Determining the model capacity.	Small units may underfit and reduce model performance. Large units may lead to overfitting and slow training.
Dropout rate	0.2, 0.3, 0.4, 0.5	A frequency of rate in dropout layer. It prevents overfitting by randomly deactivate neurons.	A low rate has a slight effect on regularization. A high rate may underfit the model by discarding too much information.
Learning rate	0.01, 0.001, 0.0001	The size of step during gradient descent moves at each iteration. Too large may overshoot minimum. To small may slow down convergence.	A low rate may cause slow training, such as getting stuck in local minima. A high rate will accelerate the convergence speed, but it has a risk of exploding gradients.

4. Results and discussion

4.1 Random Forest Classifier

After running 20 trials of hyperparameter tuning, we listed top 5 Random Forest classifier hyperparameter combinations. We find that Random Forest achieved the highest score when it had 300 decision trees. When Random Forest has 200 decision trees, the scores are similar. And when the decision tree reached 500 trees, Random Forest did not score higher score. This is consistent with our prediction that marginal returns decrease when there are too many decision trees, and underfitting is prone to occur when there are too few decision trees. The score for a decision tree depth of 30 is higher than the score for an unrestricted decision tree depth. This is also consistent with our expectations that excessive tree depth can make it easier for a single decision tree to fit noise. When min_stamples_split is 2 and min_samples_leaf is 2, Random Forest achieves the highest score. This is consistent with our prediction that when the values of min_stamples_split and min_samples_leaf are small, the capacity of Random Forest will be stronger. However, we find that the mean test score of hyperparameter combination with index 2 is lower than index 3. This is because when the values of min_samples_leaf and min_samples_split are too small, it may cause Random Forest to fit noise, resulting in a decrease in model performance. We find that Random Forest has higher accuracy on the training set than the test set. This is because Random Forest consists of several decision trees, which can fit the training data well during training.

Table 11. Top 5 Random Forest Classifier hyperparameter combinations of mean test score

n_estimators	max_depth	min_samples_split	min_samples_leaf	mean_train_score	mean_test_score	std_test_score
300	30	2	2	0.999961	0.643594	0.006087
200	30	5	2	0.999896	0.642461	0.004522
200	30	2	2	0.999957	0.641875	0.005237
500	None	10	4	0.989978	0.641836	0.003385

When the number of decision trees in Random Forest is 500 and the depth of the decision trees is not limited, Random Forest has the longest training time. The more decision trees, the longer the training time. The longer the depth of the decision tree, the longer the training time. The smaller the min_samples_leaf and min_samples_split, the longer the training time.

Table 12. Top 5 Random Forest Classifier hyperparameter combinations of training time

n_estimators	max_depth	min_samples_leaf	min_samples_split	Average Training Time (Seconds)
500	None	4	5	69.9
500	None	4	10	56.6
500	15	4	5	51.9
500	None	8	5	51.6
500	15	8	10	49.2

4.2 Multilayer Perceptron Neural Network

After running 20 trials of hyperparameter tuning, we listed top 5 MLP hyperparameter combinations. The base unit of best hyperparameter combination is 512. But the model employs strong l2 regularization and sets the dropout of the first hidden layer to 4 to prevent overfitting. Increasing the

number of neurons in MLP can improve the performance. But at the same time, it also increases the risk of overfitting. A high level of L2 regularization can suppress model complexity to mitigate overfitting. Dropout randomly blocks a portion of neurons during the training process, so that the model does not rely on specific paths, thereby reducing the risk of overfitting.

Figure 9 shows the relationship between base units and mean validation accuracy. When the number of base units increased from 128 to 256, the accuracy increased from about 0.61 to about 0.67, indicating that increasing neurons significantly improved MLP performance. When the number of base units further expanded to 320, the accuracy suddenly dropped to 0.61. By analyzing logs, we speculate that the performance degradation of the model may be due to the low rate of L2 regularization and dropout. When the number of base units increased from 320 to 384, the accuracy increased from about 0.61 to about 0.642. The accuracy decreases slightly when the base units increase from 384 to 448. Afterwards, the accuracy increased as the number of base units increased from 448 to 512.

Through the analysis of trends and logs, we find that the number of neurons increases, the L2 regularization strength and dropout rate need to be correspondingly increased. If the regularization constraint is too loose, it may lead to overfitting of the model, resulting in a decrease in performance, as shown in the two segments of decreasing accuracy trend in the figure.

Figure 10 shows that accuracy reaches its highest value when the number of base units reaches 256. Table 13 shows that the configuration with 192, 256, or 320 base units can achieve an accuracy of approximately 0.67, which is slightly different from the configuration of 512 base units. This is because when the network capacity is sufficient to express the data features, the marginal benefits of further increasing the width will significantly decrease.

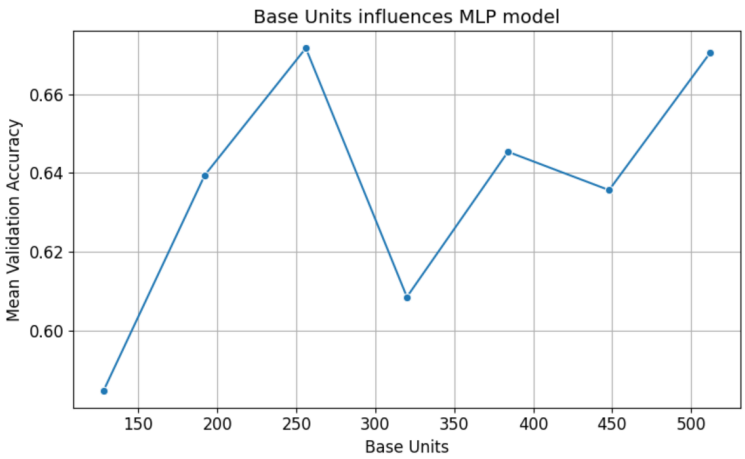

Figure 10. Units size influences MLP model

Table 13. Top 5 MLP hyperparameter combinations

base_unit	l2	dropout_0	dropout_1	dropout_2
512	0.0100	0.4	0.2	0.1
512	0.0001	0.1	0.4	0.5
192	0.0001	0.4	0.5	0.1
256	0.0010	0.2	0.1	0.4
320	0.0001	0.3	0.3	0.2

Figure 11 shows the relationship between learning rate and mean validation accuracy. When the learning rate increases from 0.0001 to 0.001, the accuracy increases from about 0.645 to about 0.652. This is because the smaller learning rate updates slowly and is prone to get stuck in local minima. The larger learning rate accelerates the convergence speed, which helps the model escape from local minima and approach the global optimum more closely, thereby enhancing the MLP's performance. When the learning rate increases from 0.001 to 0.01, the accuracy decreases from about 0.652 to about 0.59. A high learning rate leads to a wide range of weight update steps, which may cause fluctuations in the training process and make it difficult to converge to the bottom of the loss, thereby weakening the performance of the model.

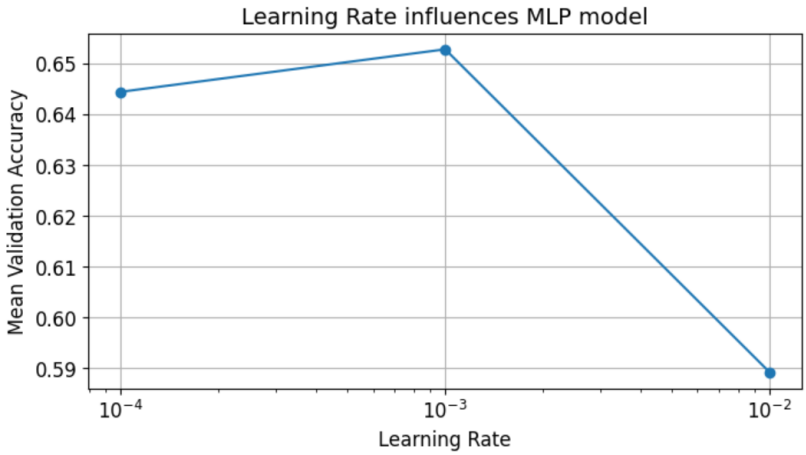

Figure 11. Learning rate influences MLP model

4.3 Convolutional Neural Networks

After running 20 trials of hyperparameter tuning, each trial run over 10 epochs with early stopping monitors validation loss. We got several best hyperparameters. Table 14 shows the 5 best hyperparameter combinations. We observed that higher base filter sizes and dense units present strong accuracy results. In addition, a larger learning rate generally leads to faster convergence. For example, small filters may not capture enough spatial patterns, while large filters may cause overfitting or extract redundant information. For the dropout rate, a lower rate may cause poor regularization, while a higher rate may result in weak performance. We need to find the best dropout rate with sufficient regularization without hindering learning capacity. For runtime, the CNN model with a higher base filter size or dense units increased each epoch's training time. The original model with a filter size of 32 took around 25 seconds to train one epoch. In contrast, the final model with a filter size of 96 took about 2 minutes to train one epoch. Selecting the best configuration while maintaining reasonable runtime and achieving high performance is important.

Table 14. Top 5 CNN hyperparameter combinations

base_unit	l2	dropout_0	dropout_1	dropout_2	learning_rate	val_accuracy
512	0.0100	0.4	0.2	0.1	0.0001	0.681406
512	0.0001	0.1	0.4	0.5	0.0010	0.680313
192	0.0001	0.4	0.5	0.1	0.0010	0.673750
256	0.0010	0.2	0.1	0.4	0.0010	0.671719
320	0.0001	0.3	0.3	0.2	0.0001	0.668594

Figure 12 shows that the filter size affects the mean validation accuracy of our CNN model. Filter size began at an accuracy of around 73% and experienced a sharp decrease to around 53% at filter sizes of about 30 and 65, respectively. Then, it increased dramatically to approximately 83% at filter size 96. This trend reflects our observations that small filters may not capture enough spatial features, while too many filters may cause overfitting or extract redundant information. The peak at a filter size of 96 reflects a great balance between feature extraction and generalization. In addition, these results aligned with our prediction that higher filter sizes would have better performance. However, I didn't expect that accuracy would drop significantly with a higher filter size. This trend highlights that the CNN model is sensitive to architecture parameters.

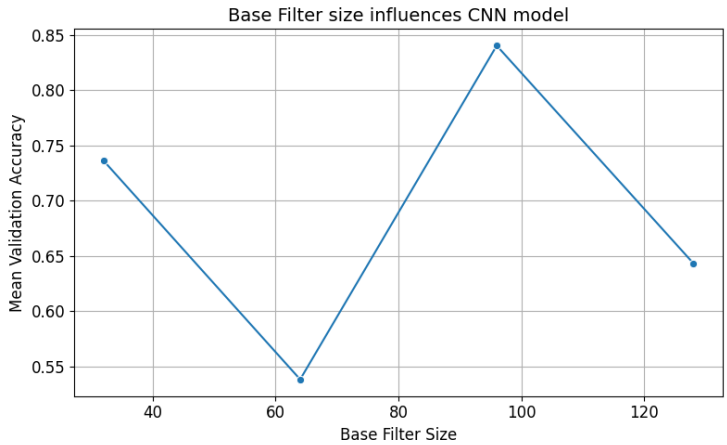

Figure 12. Filter size influences CNN model

Figure 13 indicates that the learning rate size impacts the mean validation accuracy of the CNN model. Accuracy is very high at the beginning, and it improved slightly from about 83% to 90% at learning rates 0.0001 and 0.001, respectively. However, it dropped heavily to approximately 22% at a learning rate of 0.01. This trend means a very low learning rate that which may result in low convergence, but it ensures stability. An optimal learning rate of 0.001 balances convergence speed and model performance. In contrast, a high learning rate of 0.01 caused the optimizer to overshoot minima. Then, causing poor training and low validation accuracy. These results match our predictions that a learning rate of 0.001 would yield the best performance. Overall, the results highlight the importance of tuning this hyperparameter.

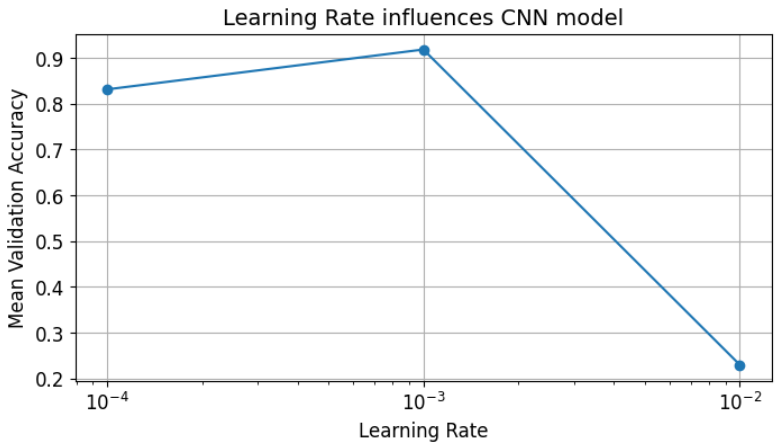

Figure 13. Learning Rate influences CNN model

Overall, the tuning results validated our theoretical understanding of CNN behaviour. Based on the previous results and observations, we have selected the best hyperparameter configurations: 96 base filters, 64 dense units, a dropout rate of 0.3, and a learning rate of 0.001. This combination has the best trade-off between model complexity, generalization, and convergence. The results aligned with our theoretical expectations based on an appropriate CNN architecture design.

4.4 Comparison of the models

Table 15 presents the outcome of each algorithm that we have performed on this task. We observed that the CNN model achieved the highest accuracy with 93.7% accuracy and weighted/macro F1-scores of 94% on test data. This trend aligns with theoretical expectations that CNNs are designed to process spatial hierarchical data like images. Because they can extract hierarchical and translation-invariant features. The accuracy of MLP is lower than that of CNN. This is because MLP flattens the image into one-dimensional vectors and inputs them into the network, which results in MLP losing the spatial local relationships between pixels in the image. Therefore, MLP cannot utilize some local patterns in images for feature extraction. MLP performed better than the Random Forest classifier with an accuracy of 68.5% and an F1-score of 69%. While the Random Forest classifier showed the lowest accuracy, which has an accuracy of 64.3% and an F1-score of 62%. The reason is that MLP trained with the backpropagation algorithm are universal approximators. Any continuous function can be approximated with arbitrary small error by a network with one hidden layer [7]. Any function can be approximated to arbitrary small error by a network with two hidden layers [8]. Although MLP flattens the image into one-dimensional vectors, resulting in the loss of some spatial structural information. It can still learn some feature representations from the original pixels due to its non-linear connections between neurons and multi-layer structure. In contrast, Random Forest treats each pixel as an independent feature and cannot understand the spatial relationships between these pixels, making it difficult to extract complex high-level semantic information from the original pixels.

For the training time, CNNs required more time to pass through deeper architecture and convolutional layers. It took over 12 minutes to finish training. The MLP model took less time than the CNN model, with about 2 minutes. However, the Random Forest classifier trained the fastest among all models. That's because MLP requires updating many weights between neurons in each layer. CNN needs to train convolutional kernels and compute many feature maps. However, Random Forest only involved tree construction without updating weights interactively.

Table 15. Model Tuning results on test set comparisons

Model	Accuracy	Macro F1	Weighted F1	Training Time	Hyperparameters
Random Forest classifier	0.6428	0.6032	0.6157	26.36 sec	n_estimator = 300, max_depth = 30, min_sample_split = 2, min_samples_leaf = 2
MLP model	0.6848	0.68	0.69	2 min 14.66 sec	base_unit = 512, l2 = 0.01, dropout_0 = 0.4, dropout_1 = 0.2, droupout_2 = 0.1, learning_rate = 0.0001
CNN model	0.9371	0.94	0.94	12 min 39.85 sec	Base_filter = 96, dense_units = 64,

					dropout_rate = 0.3, learning_rate = 0.001

Figure 14 compares the precision and recall scores of CNN and MLP models. The MLP model has poor performance on classes 2, 4, 6, 7, and 8, while the CNN model has slightly weaker performance on class 7. This trend indicates that CNN not only achieved high accuracy but also maintained robustness in both false positives and false negatives.

```
CNN Final Model - accuracy on test data: 0.9371
              precision    recall  f1-score   support

           0       0.97      0.99      0.98       873
           1       0.99      0.99      0.99       858
           2       0.92      0.86      0.89       877
           3       0.98      0.98      0.98       914
           4       0.97      0.90      0.93       737
           5       0.95      0.93      0.94      1072
           6       0.93      0.92      0.93       682
           7       0.84      0.88      0.86       813
           8       0.91      0.96      0.93      1174

    accuracy                           0.94      8000
   macro avg       0.94      0.93      0.94      8000
weighted avg       0.94      0.94      0.94      8000

CNN Final Model - training time: 759.85 seconds
```

```
MLP Final Model - accuracy on test data: 0.6848
              precision    recall  f1-score   support

           0       0.86      0.97      0.91       873
           1       0.98      0.88      0.93       858
           2       0.51      0.45      0.48       877
           3       0.93      0.87      0.90       914
           4       0.65      0.64      0.64       737
           5       0.70      0.64      0.67      1072
           6       0.46      0.47      0.46       682
           7       0.51      0.52      0.52       813
           8       0.57      0.67      0.61      1174

    accuracy                           0.68      8000
   macro avg       0.69      0.68      0.68      8000
weighted avg       0.69      0.68      0.69      8000

MLP Final Model - training time: 134.66 seconds
```

Figure 14. Classification Report of CNN and MLP model.

Figure 15 shows the CNN and MLP models' training and validation performances. Firstly, the MLP model's training and validation loss decline smoothly and steadily, which means the network is stable and keeps learning. However, the training and validation accuracy curves cross at around 13 epochs, suggesting the model begins to overfitting. Secondly, the CNN model converges faster than the MLP model, which achieved an accuracy of over 90% after 5 epochs. For the loss, its training loss decreased steadily, but the validation loss began to fluctuate after epoch 10. This trend indicates a potential overfitting, but less severe than the MLP model. Overall, the CNN model consistently achieved higher accuracy than the MLP model on both training and validation data. Thus, it again reinforces its ability to capture complex spatial patterns. Moreover, the performance plateau observed in both models indicates that the model starts applying early stopping. This helps to prevent overfitting and reduce unnecessary training time.

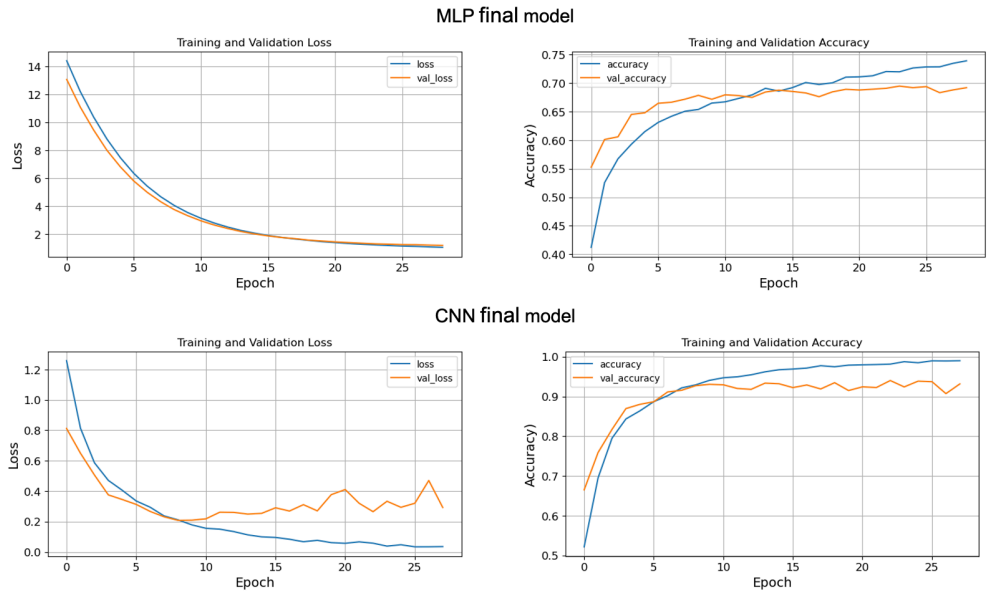

Figure 15. Training and Validation Loss and Accuracy performance of MLP and CNN model

In conclusion, the results met our theoretical expectations that the CNN model is powerful for the image classification task. The MLP model is moderate but less effective for image data. Random Forest has the benefit of speed but lacks feature learning capability.

5. Conclusion

5.1 Summary of main findings and identification of study limitations

Our classification task implements three different algorithms on the PathMNIST dataset. The results revealed that the custom CNN ("mini VGG16") model surpassed the MLP model and the Random Forest Classifier. CNN model achieved a test accuracy of 93.71% (F1 ≈ 94%). However, the MLP model was 68.48% (F1 ≈ 69%), and the Random Forest classifier was 64.28% (F1 ≈ 61.6%). According to runtime, training the CNN model required about 12 min, the MLP model required around 2 min, and the Random Forest only took about 26s. Although the CNN model achieved the highest accuracy, it has a longer training time and computational demands, which makes it hard to implement in resource-constrained environments. In contrast, Random Forest Classifier has faster training and better interpretability but less accuracy. In this way, it is suitable for preliminary analysis or edge deployment.

The performance of the CNN model is likely because its architecture is designed for image data. CNN's convolutional layers have the ability to extract spatial and hierarchical features, and make them well-aligned with the PathMNIST dataset. In contrast, the MLP model lacks spatial ability, and the Random Forest classifier can't capture spatial locality in images.

According to interpretability, Random Forest benefits from feature-subset voting but loses transparency by applying PCA. MLP and CNN models behave like black boxes, offering little insight into tissue structures, which makes it difficult to interpret their predictions.

However, there are some limitations that constrain our conclusions. Firstly, the 28×28 image resolution and strong H&E colour bias of the dataset may obscure histopathological details. Moreover, the model may tend to overfit to stain patterns rather than morphology. Secondly, we only used preprocessing techniques, including Gaussian blur and Z-score normalization. There are some other preprocessing techniques known for medical images, such as stain normalization or data augmentation. Finally, in hyperparameter tuning, due to hardware limitations, we didn't use grid search to search all combinations in the Random Forest classifier. If we use grid search, we will need to fit 10 folds for each of 81 candidates, totaling 810 fits. Instead, we used Random Search to search 20 iterations. In addition, we fixed the epochs and batch size on the CNN and MLP models because we only compare the validation accuracy of each trial.

For future work, we could explore several strategies to improve performance, such as data augmentation, stain normalizations, etc. Overall, our CNN model proved highly effective for the PathMNIST classification task. But we must consider other issues, such as interpretability, runtime, and deployment constraints. This highlights the importance of balancing performance with practical feasibility in real-world applications.

5.2 Future Work Suggestions

Based on the limitations of our models, we have proposed the following improvement suggestions:

- Data Augmentation & Stain Normalization

  Applying geometric transforms (rotations, flips, random crops) and H&E-specific colour normalization. Data Augmentation increases dataset size, which mitigates overfitting and

improves robustness to spatial variations. Stain Normalization helps models focus on tissue morphology rather than artefactual hue differences.

- Ensemble Strategies

  Combining CNN and Random Forest (or MLP) by using stacking or weighted voting. Ensembles can improve both accuracy and robustness, which compensates for individual model weaknesses and smooths out variability in decision boundaries.

- Automated Hyperparameter Optimization

  Employing Bayesian optimization frameworks to explore learning rates, architecture depths, and regularization strengths. Structured search methods can identify non-intuitive hyperparameter settings faster and more reliably than random search, especially in high-dimensional spaces.

Overall, we may improve performance, interpretability, and reliability by applying these strategies based on limitations. These improvements will help to develop more robust and generalizable models, espacially in real-world clinical settings where staining variability and limited data remain key challenges.

6. Reflection

6.1 Student SID: 540584226

The most important things that I have learned from this assignment are the critical impact of model architecture and hyperparameter tuning on performance in image classification tasks. In this assignment, I experimented with three different algorithms, including Random Forest, MLP, and CNN. As a result, I have better understanding why convolutional neural networks are suitable for handling spatial data. For example, CNNs can learn hierarchical features from images. In contrast, MLP or Random Forest fail to preserve spatial locality, leading to suboptimal results. Moreover, I realized that only relying on accuracy is insufficient. I explored other performance metrics, such as precision, recall, and F1-score. They can provide more comprehensive insight into model behaviour, especially in classification problems.

This assignment has reinforced my understanding of machine learning workflows. Starting from data preprocessing and model selection to training, evaluation, and visualization. Plotting learning curves and interpreting class-wise performance, helped me diagnose overfitting and assess model generalization. Furthermore, tuning parameters like learning rate, dropout, and filter size showed how a small adjustment can significantly affect results. Overall, this assignment solidified my theoretical knowledge through hands-on experience and improved my ability to critically analyse and compare model performance in real-world scenarios.

6.1 Student SID: 540202537

The most important thing I have learned is the importance of the preprocessing stage. At first, the preprocessing method we used was relatively simple: we normalized the pixel values by dividing them by 255. As a result, the accuracy of our MLP consistently being lower than that of the Random Forest. Even after adjusting the parameters for two more days, we were unable to make the accuracy of MLP exceed that of the random forest. After trying many hyperparameter combinations, we pre-processed the data again. Firstly, to reduce noise interference in the images, we applied Gaussian blur processing to each image in the dataset. Consequently, we adopted the channel level normalization method. After conducting these steps, we found that even though the hyperparameters of MLP did not change, its

accuracy was significantly higher and exceeded that of Random Forest. I realized that data preprocessing is an extremely important step in machine learning and deep learning. Good preprocessing can significantly improve model performance. After reviewing relevant materials, I came across the following statement: "Today's real-world databases are highly susceptible to noisy, missing, and inconsistent data due to their typically huge size (often several gigabytes or more) and their likely origin from multiple, heterogenous sources. Low-quality data will lead to low-quality mining results." [11]. The PathMNIST dataset has high quality, but in our experiments, we found that even on such a relatively clean dataset, different preprocessing strategies can have a significant impact on the performance of the MLP model. In practical applications, real-world data often suffers from problems such as noise, missing values, and inconsistent formats. Therefore, good data preprocessing can not only improve model performance, but also enhance model stability and generalization ability, and its importance cannot be ignored.

References

[1] R. D. Odze and J. R. Goldblum, *Odze and Goldblum surgical pathology of the GI tract, liver, biliary tract, and pancreas*, Third edition. Philadelphia, PA: Saunders/Elsevier, 2015.

[2] Y. Gao *et al.*, 'Application of artificial intelligence in the diagnosis of malignant digestive tract tumors: focusing on opportunities and challenges in endoscopy and pathology', *J Transl Med*, vol. 23, no. 1, p. 412, Apr. 2025, doi: 10.1186/s12967-025-06428-z.

[3] J. Yang *et al.*, 'MedMNIST v2 - A large-scale lightweight benchmark for 2D and 3D biomedical image classification', *Sci Data*, vol. 10, no. 1, p. 41, Jan. 2023, doi: 10.1038/s41597-022-01721-8.

[4] G. Litjens *et al.*, 'A survey on deep learning in medical image analysis', *Medical Image Analysis*, vol. 42, pp. 60–88, Dec. 2017, doi: 10.1016/j.media.2017.07.005.

[5] *Principal Component Analysis*. in Springer Series in Statistics. New York: Springer-Verlag, 2002. doi: 10.1007/b98835.

[6] L. Breiman, 'Random Forests', *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/A:1010933404324.

[7] K. Hornik, M. Stinchcombe, and H. White, 'Multilayer feedforward networks are universal approximators', *Neural Networks*, vol. 2, no. 5, pp. 359–366, Jan. 1989, doi: 10.1016/0893-6080(89)90020-8.

[8] G. Cybenko, 'Approximation by superpositions of a sigmoidal function', *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, Dec. 1989, doi: 10.1007/BF02551274.

[9] Y. LeCun, Y. Bengio, and G. Hinton, 'Deep learning', *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015, doi: 10.1038/nature14539.

[10] K. Simonyan and A. Zisserman, 'Very Deep Convolutional Networks for Large-Scale Image Recognition', Apr. 10, 2015, *arXiv*: arXiv:1409.1556. doi: 10.48550/arXiv.1409.1556.

[11] S. Chakrabarti *et al.*, *Data Mining: Know It All*, Online-Ausg. in EBL-Schweitzer. Burlington: Elsevier Science, 2014.